

Практичне заняття №7

Redis кешування + Query параметри + Pagination

Дисципліна: Високорівневі мови програмування та фреймворки

Мета

Дати Redis роботу: використати підключений ще на Практичній 2 Redis для кешування списку продуктів. Реалізувати pagination (page + pageSize), сортування (sort + order) та фільтрацію (categoryId, minPrice, maxPrice, search) для GET /api/products через Query параметри з валідацією. Перейти з productRepo.find() на QueryBuilder для побудови складних запитів.

Передумови

Виконане Практичне заняття №6: MiniShop API з автентифікацією, Swagger, Interceptors, Exception Filters. Redis підключений і працює (перевірка: docker compose exec redis redis-cli ping → PONG). CacheModule вже зареєстрований глобально в app.module.ts.

Що здаємо (тільки одне)

Лінк на GitHub репозиторій, в якому є:

- README.md з коротким звітом (шаблон нижче) + виводом команд
- ProductQueryDto з валідацією query параметрів
- Pagination у GET /api/products з форматом { items, meta }
- Фільтрація та сортування через query параметри
- Redis кешування з інвалідацією
- Мінімум 3 нових коміти

Важливо: GET /api/products без параметрів повинен повертати першу сторінку з мета-інформацією, а не весь масив. Необмежений pageSize заборонений — максимум 100.

Вправа 1 — Створити ProductQueryDto

Опис

Створити DTO для Query параметрів з валідацією: page, pageSize, sort, order, categoryId, minPrice, maxPrice, search. Кожен параметр необов'язковий і має

значення за замовчуванням. Це захищає від «вбивчих» запитів — клієнт не може запросити 10 000 записів або відсортувати за неіснуючим полем.

Інструкції (покроково)

1) Створити файл `src/products/dto/product-query.dto.ts`:

```
import {
  IsOptional,
  IsInt,
  Min,
  Max,
  IsIn,
  IsString,
  IsNumber,
} from 'class-validator';
import { Type } from 'class-transformer';
import { ApiPropertyOptional } from '@nestjs/swagger';

export class ProductQueryDto {
  @ApiPropertyOptional({
    example: 1,
    description: 'Номер сторінки',
    default: 1,
  })
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  page?: number = 1;

  @ApiPropertyOptional({
    example: 10,
    description: 'Кількість елементів на сторінці (max 100)',
    default: 10,
  })
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  @Min(1)
  @Max(100)
  pageSize?: number = 10;

  @ApiPropertyOptional({
    example: 'price',
    description: 'Поле для сортування',
    enum: ['name', 'price', 'stock', 'createdAt'],
    default: 'createdAt',
  })
```

```
    })
    @IsOptional()
    @IsIn(['name', 'price', 'stock', 'createdAt'])
    sort?: string = 'createdAt';
```

```
    @ApiPropertyOptional({
        example: 'desc',
        description: 'Напрямок сортування',
        enum: ['asc', 'desc'],
        default: 'desc',
    })
```

```
    @IsOptional()
    @IsIn(['asc', 'desc'])
    order?: 'asc' | 'desc' = 'desc';
```

```
    @ApiPropertyOptional({
        example: 1,
        description: 'Фільтр за ID категорії',
    })
```

```
    @IsOptional()
    @Type(() => Number)
    @IsInt()
    categoryId?: number;
```

```
    @ApiPropertyOptional({
        example: 100,
        description: 'Мінімальна ціна',
    })
```

```
    @IsOptional()
    @Type(() => Number)
    @IsNumber()
    @Min(0)
    minPrice?: number;
```

```
    @ApiPropertyOptional({
        example: 1000,
        description: 'Максимальна ціна',
    })
```

```
    @IsOptional()
    @Type(() => Number)
    @IsNumber()
    @Min(0)
    maxPrice?: number;
```

```
    @ApiPropertyOptional({
        example: 'iPhone',
```

```

        description: 'Пошук за назвою (ILIKE)',
    })
    @IsOptional()
    @IsString()
    search?: string;
}

```

Пояснення ключових моментів

Елемент	Навіщо
@Type(() => Number)	Query параметри завжди приходять як рядки. @Type конвертує "10" → 10 перед валідацією. Без нього @IsInt() завжди буде fallити.
@Max(100) на pageSize	Захист від «вбивчих» запитів. ?pageSize=10000 поверне 400, а не повалить базу.
@IsIn(['name','price'...])	Allow-list для сортування. Клієнт не може відсортувати за passwordHash або іншим внутрішнім полем.
default: 'createdAt'	Якщо клієнт не передав sort — сортуємо за датою створення. Стабільне сортування за замовчуванням.

Очікуваний результат

- ProductQueryDto створений з повною валідацією та дефолтами
- Swagger показує всі query параметри з описами та прикладами

Вправа 2 — Оновити ProductService: QueryBuilder + Pagination

Опис

Замінити productRepo.find() на createQueryBuilder для побудови динамічних запитів з пагінацією, сортуванням та фільтрацією. Сервіс повертає { items, meta } замість простого масиву.

Інструкції (покроково)

1) Оновити метод findAll у src/products/products.service.ts:

```
import { ProductQueryDto } from '../dto/product-query.dto';
```

// Замінити старий findAll на:

```
async findAll(query: ProductQueryDto) {
  const {
    page = 1,
    pageSize = 10,
    sort = 'createdAt',
    order = 'desc',
    categoryId,
    minPrice,
    maxPrice,
    search,
  } = query;

  const qb = this.productRepo
    .createQueryBuilder('product')
    .leftJoinAndSelect(
      'product.category',
      'category',
    );

  // Фільтр за категорією
  if (categoryId) {
    qb.andWhere(
      'category.id = :categoryId',
      { categoryId },
    );
  }

  // Фільтр за ціною
  if (minPrice !== undefined) {
    qb.andWhere(
      'product.price >= :minPrice',
      { minPrice },
    );
  }
  if (maxPrice !== undefined) {
    qb.andWhere(
      'product.price <= :maxPrice',
      { maxPrice },
    );
  }

  // Пошук за назвою (case-insensitive)
  if (search) {
```

```

        qb.andWhere(
            'product.name ILIKE :search',
            { search: `%${search}%` },
        );
    }

    // Сортювання
    qb.orderBy(
        `product.${sort}`,
        order.toUpperCase() as 'ASC' | 'DESC',
    );

    // Парінація
    const skip = (page - 1) * pageSize;
    qb.skip(skip).take(pageSize);

    // Виконати запит з підрахунком
    const [items, total] = await qb.getManyAndCount();

    return {
        items,
        meta: {
            page,
            pageSize,
            total,
            totalPages: Math.ceil(total / pageSize),
        },
    };
}

```

Пояснення QueryBuilder

Метод	SQL-еквівалент
createQueryBuilder('product')	FROM products AS product
leftJoinAndSelect('product.category', 'category')	LEFT JOIN categories AS category ON ...
andWhere('product.price >= :minPrice', { minPrice })	AND product.price >= \$1 (параметризований запит)
ILIKE :search, `%\${search}%`	WHERE product.name ILIKE '%iphone%' (case-insensitive пошук)
skip(10).take(10)	OFFSET 10 LIMIT 10

getManyAndCount()	Два запити: SELECT ... та SELECT COUNT(*) — повертає [items, total]
-------------------	---

💡 *Параметризовані запити (:minPrice) автоматично захищають від SQL-ін'єкцій. НІКОЛИ не підставляйте значення напряду в рядок запиту через `template literals`.*

Очікуваний результат

- `findAll` приймає `ProductQueryDto` і повертає `{ items, meta }`
- Підтримує пагінацію, сортування, фільтрацію та пошук

Вправа 3 — Оновити ProductsController

Опис

Оновити контролер: метод `findAll` приймає `@Query()` з `ProductQueryDto`. Swagger автоматично покаже всі query параметри з описами.

Інструкції (покроково)

1) Оновити метод `findAll` у `src/products/products.controller.ts`:

```
import { Query } from '@nestjs/common';
import { ProductQueryDto }
  from '../dto/product-query.dto';

@Get()
@ApiOperation({
  summary: 'Отримати продукти з пагінацією',
  description:
    'Повертає список продуктів з мета-інформацією. ' +
    'Підтримує пагінацію, сортування, фільтрацію ' +
    'та пошук.',
})
findAll(@Query() query: ProductQueryDto) {
  return this.productsService.findAll(query);
}
```

2) Перезібрати та перевірити:

```
docker compose up --build -d
```

```
# Базовий запит (перша сторінка, 10 елементів)
curl http://localhost:3000/api/products
```

```
# Друга сторінка, 5 елементів
```

```
curl "http://localhost:3000/api/products?page=2&pageSize=5"

# Сортуння за ціною
curl "http://localhost:3000/api/products?sort=price&order=asc"

# Фільтр за ціною
curl "http://localhost:3000/api/products?minPrice=100&maxPrice=500"

# Пошук
curl "http://localhost:3000/api/products?search=iPhone"

# Комбінований запит
curl
"http://localhost:3000/api/products?categoryId=1&minPrice=500&sort=price&order=asc&
page=1&pageSize=5"
```

Очікується формат відповіді (всередині data від TransformInterceptor):

```
{
  "data": {
    "items": [ ... масив продуктів ... ],
    "meta": {
      "page": 1,
      "pageSize": 10,
      "total": 47,
      "totalPages": 5
    }
  },
  "statusCode": 200,
  "timestamp": "..."
}
```

Очікуваний результат

- GET /api/products повертає { items, meta } з пагінацією
- Query параметри видно у Swagger UI
- Невалідні параметри повертають 400 (наприклад, ?pageSize=999)

Вправа 4 — Додати Redis кешування до GET /api/products

Опис

Використати CACHE_MANAGER (підключений ще на Практичній 2) для кешування результатів GET /api/products у Redis. Ключ кешу формується з query параметрів — різні фільтри зберігають різні кеші. TTL — 60 секунд.

Інструкції (покроково)

1) Оновити src/products/products.service.ts — додати CACHE_MANAGER:

```
import { Injectable, NotFoundException, Inject }
  from '@nestjs/common';
import { CACHE_MANAGER } from '@nestjs/cache-manager';
import { Cache } from 'cache-manager';

@Injectable()
export class ProductsService {
  constructor(
    @InjectRepository(Product)
    private readonly productRepo: Repository<Product>,
    @Inject(CACHE_MANAGER)
    private readonly cacheManager: Cache,
  ) {}

  async findAll(query: ProductQueryDto) {
    // 1. Сформувати ключ кешу з query
    const cacheKey = `products:${JSON.stringify(query)}`;

    // 2. Перевірити кеш
    const cached = await this.cacheManager.get(cacheKey);
    if (cached) {
      return cached;
    }

    // 3. Виконати запит до БД
    const { page = 1, pageSize = 10, sort = 'createdAt',
      order = 'desc', categoryId, minPrice,
      maxPrice, search } = query;

    const qb = this.productRepo
      .createQueryBuilder('product')
      .leftJoinAndSelect('product.category', 'category');

    if (categoryId) {
      qb.andWhere('category.id = :categoryId',
        { categoryId });
    }

    if (minPrice !== undefined) {
      qb.andWhere('product.price >= :minPrice',
```

```

    { minPrice });
  }
  if (maxPrice !== undefined) {
    qb.andWhere('product.price <= :maxPrice',
    { maxPrice });
  }
  if (search) {
    qb.andWhere('product.name ILIKE :search',
    { search: `%${search}%` });
  }

  qb.orderBy(`product.${sort}`,
  order.toUpperCase() as 'ASC' | 'DESC');

  const skip = (page - 1) * pageSize;
  qb.skip(skip).take(pageSize);

  const [items, total] = await qb.getManyAndCount();

  const result = {
    items,
    meta: { page, pageSize, total,
    totalPages: Math.ceil(total / pageSize) },
  };

  // 4. Зберегти в кеш (TTL 60 секунд)
  await this.cacheManager.set(
  cacheKey, result, 60_000,
  );

  return result;
}

// ... решта методів ...
}

```

 **Ключ кешу `products:${JSON.stringify(query)}` — це простий але ефективний підхід. Різні комбінації фільтрів створюють різні ключі. Якщо клієнт змінив `pageSize` — це інший кеш.**

2) Перевірити що кеш працює:

```
docker compose restart app
```

```
# Перший запит — з бази (повільніше)
curl "http://localhost:3000/api/products"
```

```
# Другий запит — з кешу (швидше)
curl "http://localhost:3000/api/products"
```

```
# Перевірити ключі в Redis
docker compose exec redis redis-cli KEYS "products:*"
```

Очікується: `redis-cli KEYS` показує один або кілька ключів з префіксом `products:`.

Очікуваний результат

- Повторні GET-запити з однаковими параметрами повертають дані з Redis (швидше)
- `redis-cli KEYS` показує ключі кешу
- Через 60 секунд кеш протухає і наступний запит йде в базу

Вправа 5 — Інвалідація кешу при зміні даних

Опис

Коли продукт створюється, оновлюється або видаляється — кеш стає неактуальним. Потрібно очищати всі ключі з префіксом `products:` після будь-якої мутації. Без інвалідації клієнт буде бачити старі дані до 60 секунд після зміни.

Інструкції (покроково)

1) Додати приватний метод очищення кешу в `ProductsService`:

```
private async clearProductsCache(): Promise<void> {
  const keys: string[] = await this.cacheManager
    .store.keys('products:*');
  if (keys.length > 0) {
    await Promise.all(
      keys.map((key) => this.cacheManager.del(key)),
    );
  }
}
```

2) Викликати `clearProductsCache()` у методах `create`, `update` та `remove`:

```
async create(dto: CreateProductDto): Promise<Product> {
  const product = this.productRepo.create({ ... });
  const saved = await this.productRepo.save(product);
  await this.clearProductsCache(); // <-- додати
  return saved;
}
```

```

async update(id: number, dto: UpdateProductDto)
  : Promise<Product> {
  const product = await this.findOne(id);
  // ... оновлення полів ...
  const saved = await this.productRepo.save(product);
  await this.clearProductsCache(); // <-- додати
  return saved;
}

async remove(id: number): Promise<void> {
  const product = await this.findOne(id);
  await this.productRepo.remove(product);
  await this.clearProductsCache(); // <-- додати
}

```

3) Перезапустити та перевірити інвалідацію:

```
docker compose restart app
```

```
# 1. Запит — дані потрапляють у кеш
```

```
curl "http://localhost:3000/api/products"
```

```
# 2. Перевірити кеш
```

```
docker compose exec redis redis-cli KEYS "products:*"
```

```
# Очікується: 1 ключ
```

```
# 3. Створити новий продукт (з admin-токеном)
```

```

curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <ADMIN_TOKEN>" \
  -d '{"name": "New Product", "price": 99.99}'

```

```
# 4. Перевірити кеш — має бути порожній
```

```
docker compose exec redis redis-cli KEYS "products:*"
```

```
# Очікується: (empty array)
```

```
# 5. Наступний GET — свіжі дані з бази
```

```
curl "http://localhost:3000/api/products"
```

⚠ Метод `store.keys()` доступний через `cache-manager-redis-yet`. Якщо ви використовуєте інший `store` — API може відрізнятись. Перевірте документацію вашого `cache store`.

Очікуваний результат

- Після POST/PATCH/DELETE кеш продуктів очищається
- Наступний GET повертає свіжі дані з бази, а не старий кеш

- redis-cli KEYS підтверджує очищення

Вправа 6 — Seed-скрипт для тестових даних

Опис

Pagination та фільтрація мають сенс тільки коли в базі є достатньо даних. Створити простий seed-скрипт, який додає 3 категорії та 30 продуктів із різними цінами та категоріями.

Інструкції (покроково)

1) Створити файл src/seeds/seed.ts:

```
mkdir -p src/seeds
```

```
import { DataSource } from 'typeorm';
import * as dotenv from 'dotenv';

dotenv.config();

const ds = new DataSource({
  type: 'postgres',
  host: process.env.POSTGRES_HOST,
  port: parseInt(process.env.POSTGRES_PORT, 10),
  username: process.env.POSTGRES_USER,
  password: process.env.POSTGRES_PASSWORD,
  database: process.env.POSTGRES_DB,
});

async function seed() {
  await ds.initialize();

  // Категорії
  const cats = ['Electronics', 'Accessories', 'Clothing'];
  for (const name of cats) {
    await ds.query(
      `INSERT INTO categories (name) VALUES ($1)
      ON CONFLICT DO NOTHING`,
      [name],
    );
  }

  // Продукти
  const products = [
```

```

    { name: 'iPhone 16', price: 999, stock: 50, cat: 1 },
    { name: 'Galaxy S24', price: 849, stock: 40, cat: 1 },
    { name: 'MacBook Pro', price: 2499, stock: 15, cat: 1 },
    { name: 'iPad Air', price: 599, stock: 30, cat: 1 },
    { name: 'AirPods Pro', price: 249, stock: 100, cat: 2 },
    { name: 'USB-C Cable', price: 19, stock: 500, cat: 2 },
    { name: 'MagSafe Charger', price: 39, stock: 80, cat: 2 },
    { name: 'Laptop Sleeve', price: 49, stock: 60, cat: 2 },
    { name: 'T-Shirt Dev', price: 25, stock: 200, cat: 3 },
    { name: 'Hoodie NestJS', price: 55, stock: 75, cat: 3 },
  ];

  // Додати 30 записів (3 рази по 10)
  for (let i = 0; i < 3; i++) {
    for (const p of products) {
      const suffix = i > 0 ? `v${i + 1}` : '';
      await ds.query(
        `INSERT INTO products
          (name, price, stock, "categoryId")
        VALUES ($1, $2, $3, $4)
        ON CONFLICT DO NOTHING`,
        [`${p.name}${suffix}`, p.price + i * 10,
        p.stock, p.cat],
      );
    }
  }

  console.log('Seed complete: 3 categories, 30 products');
  await ds.destroy();
}

seed().catch(console.error);

```

2) Додати npm-скрипт у package.json:

```

"scripts": {
  "seed": "ts-node src/seeds/seed.ts",
  ... інші скрипти ...
}

```

3) Запустити seed:

```
docker compose run --rm app npm run seed
```

Очікується: "Seed complete: 3 categories, 30 products".

4) Перевірити пагінацію з реальними даними:

```
# Перша сторінка
curl "http://localhost:3000/api/products?page=1&pageSize=5"
# Очікується: 5 items, meta.total = 30, totalPages = 6

# Остання сторінка
curl "http://localhost:3000/api/products?page=6&pageSize=5"
# Очікується: 5 items або менше

# Фільтрація + пагінація
curl "http://localhost:3000/api/products?categoryId=1&sort=price&order=desc"
```

Очікуваний результат

- 30 продуктів у 3 категоріях додані в базу
- Пагінація, фільтрація та сортування працюють з реальними даними

Вправа 7 — Тестування повного flow

Опис

Комплексне тестування: пагінація, фільтри, кешування, інвалідація, валідація query, Swagger UI.

Тести Pagination

1. Базовий запит:

```
curl "http://localhost:3000/api/products"
```

Очікується: 10 items (pageSize за замовчуванням), meta.total = 30.

2. Конкретна сторінка:

```
curl "http://localhost:3000/api/products?page=3&pageSize=5"
```

Очікується: 5 items, meta.page = 3.

3. Невалідний pageSize:

```
curl "http://localhost:3000/api/products?pageSize=999"
```

Очікується: 400 Bad Request, "pageSize must not be greater than 100".

Тести фільтрації

4. Фільтр за категорією:

```
curl "http://localhost:3000/api/products?categoryId=2"
```

Очікується: тільки продукти з категорії Accessories.

5. Діапазон цін:

```
curl "http://localhost:3000/api/products?minPrice=100&maxPrice=1000"
```

Очікується: продукти з ціною від 100 до 1000.

6. Пошук:

```
curl "http://localhost:3000/api/products?search=mac"
```

Очікується: MacBook Pro (та варіанти). ILIKE — реєстронезалежний.

Тести кешування

7. Перевірити кеш через Redis CLI:

```
# Запит — створює кеш
```

```
curl "http://localhost:3000/api/products?page=1&pageSize=5"
```

```
# Перевірити TTL
```

```
docker compose exec redis redis-cli KEYS "products:*"
```

```
docker compose exec redis redis-cli TTL "products:{\"page\":1,\"pageSize\":5}"
```

```
# Очікується: число від 0 до 60
```

8. Створити продукт та перевірити інвалідацію:

```
# Створити (потрібен admin-токен)
```

```
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <ADMIN_TOKEN>" \
  -d '{"name": "Fresh Product", "price": 42}'
```

```
# Кеш очищено
```

```
docker compose exec redis redis-cli KEYS "products:*"
```

```
# Очікується: (empty array)
```

Тест Swagger UI

9. Відкрити <http://localhost:3000/api/docs>:

GET /api/products тепер показує всі query параметри: page, pageSize, sort, order, categoryId, minPrice, maxPrice, search. Натиснути "Try it out" та виконати запит з фільтрами.

Вправа 8 — Оновити README та зробити коміти

Інструкції (покроково)

1) Зробити коміти (мінімум 3):

```
git add src/products/dto/product-query.dto.ts
git add src/products/products.service.ts src/products/products.controller.ts
git commit -m "add pagination, sorting, filtering with QueryBuilder"
```

```
git add src/products/products.service.ts
git commit -m "add Redis caching with cache invalidation"
```

```
git add src/seeds/ package.json
git commit -m "add seed script for test data"
```

```
git add README.md
git commit -m "update readme for practical 7"
```

```
git push
```

2) Оновити README.md за шаблоном:

```
## Student
- Name: <ПІБ>
- Group: <Група>
```

```
## Практичне заняття №7 – Redis + Pagination + Filtering
```

```
### Запуск проекту
```bash
cp .env.example .env
docker compose up --build
docker compose run --rm app npm run seed
```
```

```
### API: GET /api/products
```

| Параметр | Тип | Default | Опис |
|------------|----------|-----------|---------------------------------|
| page | number | 1 | Номер сторінки |
| pageSize | number | 10 | Елементів на сторінку (max 100) |
| sort | string | createdAt | Поле сортування |
| order | asc/desc | desc | Напрямок |
| categoryId | number | - | Фільтр за категорією |
| minPrice | number | - | Мінімальна ціна |
| maxPrice | number | - | Максимальна ціна |
| search | string | - | Пошук за назвою (ILIKE) |

```
### Тест пагінації
```

```

```text
<вивід curl GET /api/products?page=1&pageSize=5>
```

### Тест фільтрації
```text
<вивід curl GET /api/products?categoryId=1&minPrice=500>
```

### Тест пошуку
```text
<вивід curl GET /api/products?search=mac>
```

### Тест кешування (Redis)
```text
<вивід docker compose exec redis redis-cli KEYS "products:*">
```

### Тест інвалідації кешу
```text
<Redis KEYS до та після POST /api/products>
```

```

Troubleshooting — типові проблеми

1) @Type(() => Number) не працює — query завжди рядки

Симптоми: @IsInt() повертає помилку навіть для ?page=1.

Що зробити:

- Перевірити, що transform: true увімкнено в ValidationPipe (main.ts)
- Перевірити, що class-transformer встановлений
- @Type(() => Number) обов'язковий для кожного числового query параметра

2) "store.keys is not a function"

Симптоми: помилка при виклику clearProductsCache().

Що зробити:

- Метод keys() доступний через cache-manager-redis-yet (встановлений на Практичній 2)

- Перевірити: `docker compose run --rm app npm ls cache-manager-redis-yet`
- Якщо відсутній — встановити: `npm install cache-manager-redis-yet`

3) Кеш не очищується після POST/PATCH/DELETE

Симптоми: після створення продукту GET повертає старий список.

Що зробити:

- Перевірити, що `clearProductsCache()` викликається ПІСЛЯ `save/remove` (не перед)
- Перевірити логи на помилки Redis
- Ручна перевірка: `docker compose exec redis redis-cli FLUSHDB` (очищує ВСЕ — тільки для дебагу)

4) ILIKE не працює — пошук не знаходить нічого

Симптоми: `?search=iphone` повертає порожній масив.

Що зробити:

- ILIKE — це PostgreSQL-специфічна функція (case-insensitive LIKE). В інших БД використовуйте LOWER()
- Перевірити, що оператор пошуку обгорнутий у `%`: ``%${search}%``
- Перевірити що у продуктах є відповідні назви: `docker compose exec postgres psql -U nestuser -d nestdb -c "SELECT name FROM products;"`

5) seed.ts не запускається — connection refused

Симптоми: помилка підключення при `npm run seed`.

Що зробити:

- `seed.ts` потрібно запускати ВСЕРЕДИНІ Docker: `docker compose run --rm app npm run seed`
- Не запускайте `seed` локально (`npm run seed`) — він не має доступу до контейнера `postgres`
- Перевірити, що `postgres` запущений: `docker compose ps`